Your Learning Journey: Week 2 Practical Session

Dear Student,

You have just completed a practical session that covered some of the most fundamental concepts in Java programming. You didn't just read about variables, data types, and operators - you **used them** to build a working interactive calculator.

This document will help you understand exactly what you have learned, why it matters, and how it connects to your future as a developer.

Read through this carefully. You might be surprised by how much you already know!

- **Your Achievement Map**

Here is a visual summary of everything you have accomplished:



- **Learning Outcome 1: Choose appropriate data types for variables based on problem requirements**

What You Did:

In Task 1, you created DataTypesDemo.java and used **all eight primitive data types** in Java:

Data Type	Example You Used	Purpose
byte	studentAge = 20	Small whole numbers
short	yearOfBirth = 2006	Medium whole numbers
int	population = 54000000	Standard whole numbers
long	worldPopulation = 8000000000L	Very large whole numbers
float	interestRate = 7.5f	Less precise decimals
double	accountBalance = 15000.75	Precise decimals
char	grade = 'A'	Single characters
boolean	isActive = true	True/false values

You also used String for text:

String studentName = "Alice Wanjiku" // which is a Text (sequence of characters)

What You Learned:

Scenario	Correct Data Type	Why
A person's age	Int	Whole number, no decimals needed
A bank balance	double	Requires decimal precision (e.g., 1500.50)
A single letter grade	Char	Single character only
Whether a customer is active	boolean	Only two states: true or false
A customer's name	String	Text that can include spaces
World population	Long	Exceeds the range of int

Why This Matters:

Choosing the right data type is **critical** because:

- **Wrong choice = bugs** -Using int for money loses decimal precision
- **Wrong choice = memory waste** - Using double for age wastes memory
- **Wrong choice = crashes** - Using byte for population causes overflow

In fintech applications like **M-Pesa**:

- Transaction amounts must be double (or BigDecimal for precise money)
- Account numbers are String (they contain leading zeros)
- Customer names are String
- Transaction status is boolean (successful/failed)

What You Can Now Do:

You can look at any variable and choose the most appropriate data type for its purpose, explaining why other types would be less suitable.

- **Learning Outcome 2: Predict the output of expressions involving operator precedence and type conversion**

What You Did:

In Task 2, you created OperatorsDemo.java and explored:

1. **Arithmetic operators** (+, -, *, /, %)
2. **Relational operators** (==, !=, >, <, >=, <=)
3. **Logical operators** (&&, ||, !)
4. **Increment/decrement operators** (++, --)
5. **Operator precedence** (which operator runs first)

What You Learned:

Arithmetic Operators

Operator	Example	Result	Explanation
+	17 + 5	22	Addition
-	17 - 5	12	Subtraction
*	17 * 5	85	Multiplication
/	17 / 5	3	Integer division (decimal truncated)
%	17 % 5	2	Remainder (17 ÷ 5 = 3 remainder 2)

Operator Precedence

Expression	Step-by-Step	Final Result
10 + 5 * 2	5 * 2 = 10, then 10 + 10	**20**
(10 + 5) * 2	10 + 5 = 15, then 15 * 2	**30**
20 % 3 + 4 * 2	20 % 3 = 2, 4 * 2 = 8, 2 + 8	**10**

Precedence Order (Highest to Lowest):

1. Parentheses ()
2. Postfix ++, --
3. Prefix ++, --, !
4. Multiplication, division, modulus *, /, %
5. Addition, subtraction +, -
6. Relational <, >, <=, >=
7. Equality ==, !=
8. Logical AND &&
9. Logical OR ||
10. Assignment =

Increment/Decrement Operators

Expression	Meaning	Example (counter = 5)	Result
counter++	Use current value, then increment	postIncrement = counter++	postIncrement = 5, counter = 6
++counter	Increment first, then use new value	preIncrement = ++counter	counter = 6, preIncrement = 6

Why This Matters:

Operator precedence is a common source of bugs. If you don't understand it, your calculations will be wrong. In fintech:

- Interest calculations: principal * rate * time / 100
- Commission: amount * rate / 100
- Tax: (price + shipping) * taxRate

If you get the precedence wrong, you might calculate the wrong amount – and that's a serious problem in financial systems.

What You Can Now Do:

You can look at any expression and predict its output by applying operator precedence rules. You can also add parentheses to make your intentions clear.

- **Learning Outcome 3: Analyze a program to identify potential type mismatch errors**

What You Did:

In Task 3, you created TypeConversion.java and explored:

1. **Implicit casting** (widening) -automatic conversion
2. **Explicit casting** (narrowing) -manual conversion
3. **Type mismatch errors** - what happens when types don't match

What You Learned:

Implicit Casting (Widening) - Automatic

From	To	Example	Result
int	double	double d = 100;	100.0
int	long	long l = 100;	100
float	double	double d = 3.14f;	3.14

No data loss occurs. The smaller type fits inside the larger type.

Explicit Casting (Narrowing) – Manual

From	To	Example	Result
double	int	int i = (int) 3.14159;	3 (decimal truncated)
double	float	float f = (float) 123.456789;	123.45679 (precision lost)

Data loss occurs. The larger type must be forced into the smaller type.

Type Mismatch Errors

Error	Why It's Wrong	Fix
int x = 3.14;	Cannot assign double to int without cast	int x = (int) 3.14;
int y = "100";	Cannot assign String to int	int y = Integer.parseInt("100");
int z = true;	Cannot assign boolean to int	Use if statement
String s = 'A';	Cannot assign char to String	String s = "A"; or String s = Character.toString('A');

String Conversion

Conversion	Method	Example
Number to String	Concatenation	String s = "" + 42;
String to int	Integer.parseInt()	int i = Integer.parseInt("123");

Conversion	Method	Example
String to double	Double.parseDouble()	double d = Double.parseDouble("45.67");

Why This Matters:

Type errors are the most common compilation errors. Learning to identify them quickly saves time and frustration. In fintech:

- Reading data from files (String → double)
- Validating user input (String → int)
- Storing data in databases (converting between types)

What You Can Now Do:

You can look at any code snippet and identify potential type mismatch errors before compilation. You can also choose the correct conversion method when needed.

- **Learning Outcome 4: Construct a simple interactive program that accepts user input using Scanner and performs calculations**

What You Did:

In Task 4, you created InteractiveCalculator.java – a fully working program that:

1. Asks the user for their name
2. Asks for two numbers
3. Performs calculations (addition, subtraction, multiplication, division, modulus)
4. Displays the results with proper formatting
5. Handles edge cases (division by zero)

What You Learned:

Scanner Methods

Method	Reads	Example
nextLine()	Entire line (including spaces)	String name = scanner.nextLine();
next()	Next token (word)	String word = scanner.next();
nextInt()	Integer value	int num = scanner.nextInt();
nextDouble()	Double value	double num = scanner.nextDouble();
nextBoolean()	Boolean value	boolean flag = scanner.nextBoolean();

Program Structure

java

```
import java.util.Scanner;          // 1. Import Scanner


public class InteractiveCalculator { // 2. Class declaration
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in); // 3. Create Scanner


        // 4. Read inputs
        System.out.print("Enter your name: ");
        String name = scanner.nextLine();


        // 5. Perform calculations
        double result = num1 + num2;
```

```
        // 6. Display results
        System.out.println("Result: " + result);

        // 7. Close Scanner
        scanner.close();
    }
}
```

Why This Matters:

Interactive programs are the foundation of all software. Every app you use (M-Pesa, banking apps, social media) takes input from users, processes it, and returns output. You have now built this from scratch!

What You Can Now Do:

You can create a program that interacts with users, reads their input, performs calculations, and displays formatted results.

- **Your Progress: Where You Started vs. Where You Are Now**

Before Week 2	After Week 2
Didn't know the difference between int and double	Can choose the right data type for any scenario
Didn't understand operator precedence	Can predict output of any expression
Didn't know about type casting	Can perform implicit and explicit casting
Had never used Scanner	Can build an interactive calculator

Before Week 2	After Week 2
Didn't understand type mismatch errors	Can identify and fix them

- **How to Prove You've Achieved These Outcomes**

Here are the **artifacts** (proof) you now have:

1. **GitHub Repository:** https://github.com/YOUR-USERNAME/java-course
2. **Four Java Files in** week2**:**
    - DataTypesDemo.java
    - OperatorsDemo.java
    - TypeConversion.java
    - InteractiveCalculator.java
3. **Commit History:** Shows your progress
4. **Reflection Notes:** Your written answers to questions

These artifacts are your portfolio pieces. Employers can see them!

- **How This Connects to Your Career**

Learning Outcome	Career Application
Choose appropriate data types	In job interviews, you'll be asked to design variables for a system. You now understand this.
Predict operator output	Debugging code often involves tracing calculations. You can do this.
Identify type mismatch errors	Reading error messages and fixing them is a daily developer task.

Learning Outcome	Career Application
Build interactive programs	Every software project involves user input. You can build it.
Git & GitHub	Every employer expects you to know Git. You have a growing portfolio.

- **Student Reflection: In Your Own Words**

Answer these questions in your notebook. Be honest – this helps you learn:

1. Which data type was most confusing at first? Why?
2. What was the most challenging part of this practical session?
3. What error message did you encounter, and how did you solve it?
4. In your own words, what is the difference between implicit and explicit casting?
5. How will Scanner be useful in your future Java programs?
6. What is one thing you want to learn more about?

Your have Achieved the following:

Key Highlights

- **Topic: Variables, Data Types & Operators**

✓ **Used all 8 primitive data types**

✓ **Applied arithmetic, relational, and logical operators**

✓ **Understood operator precedence**

✓ **Performed implicit and explicit type casting**

✓ **Built an interactive calculator using Scanner**

✓ **Pushed Week 2 code to GitHub**

You are now a Java Developer

➢ What's Next?

In **Week 3**, you will:

1. Learn about **control flow** (if-else, switch)
2. Write programs that **make decisions**
3. Build a **grade calculator** and **menu-driven system**
4. Continue pushing your code to GitHub

But before you move on, make sure you can answer YES to these questions:

- □ I can choose the right data type for any variable.
- □ I can predict the output of expressions with multiple operators.
- □ I can identify and fix type mismatch errors.
- □ I can read user input using Scanner.
- □ I can build an interactive program.
- □ I have pushed my Week 2 code to GitHub.
- □ I am excited to learn more!

If you answered YES to all, you are ready for Week 3!

➢ Key Terms Glossary

Term	Definition
Data Type	Defines what kind of data a variable can hold
Primitive Type	Basic data type built into Java (int, double, char, boolean)
Reference Type	Data type that refers to an object (String, ArrayList)

Term	Definition
Operator	Symbol that performs an operation on values
Operand	Value that an operator acts upon
Precedence	Order in which operators are evaluated
Implicit Casting	Automatic type conversion (widening)
Explicit Casting	Manual type conversion (narrowing)
Scanner	Class used to read user input
Type Mismatch	Error when assigning incompatible types

- **Final Words**

"The expert in anything was once a beginner." – Helen Hayes

You started this week not knowing the difference between int and double. You end this week with:

- **Four working programs**
- **A GitHub portfolio that's growing**
- **A clear understanding of data types, operators, and input**
- **The confidence to build interactive applications**

This is just the beginning. Keep going!

END OF ACHIEVEMENT GUIDE FOR WEEK 2

"Every great developer you know got there by solving problems they were unqualified to solve until they actually did it." – Patrick McKenzie

Instructor Note: Please keep the document as it will be used as a reference and to fill in the reflection questions. This helps consolidate your learning and gives you a tangible sense of accomplishment. Use the self-assessment checklist to see to it that you have accomplished what is expected before moving to Week 3.